Guía Metodológica para la Resolución del Ejercicio 1

Patrón de Acceso a Datos para un Módulo de Gestión de Empleados

Ingeniería de Software

1. Introducción

Esta guía tiene como propósito acompañarte paso a paso en el razonamiento necesario para resolver el Ejercicio 1 de la clase práctica. No se trata solo de escribir código, sino de **comprender por qué** se elige una solución arquitectónica sobre otra. Aprenderás a analizar el contexto, seleccionar el patrón adecuado, implementarlo de forma coherente y, finalmente, evaluar sus limitaciones cuando el sistema evoluciona.

2. Fase 1: Análisis del Contexto (¿Qué necesito resolver?)

Paso 1: Descomponer el enunciado

Antes de escribir una sola línea de código, debes extraer las **palabras clave** del enunciado y traducirlas a restricciones de diseño.
Análisis del texto:

- **"Módulo de gestión de empleados"**: El dominio está centrado en una única entidad principal (`Employee`).
- **"Operaciones fundamentales de creación, lectura, actualización y eliminación (CRUD)"**: No hay operaciones complejas ni consultas analíticas. Es un CRUD básico.
- **"La lógica de negocio es elemental"**: No hay reglas de validación complejas, cálculos elaborados ni invariantes de dominio que requieran un modelo de objetos rico.
- **"No existen relaciones de agregación o composición con otras entidades"**: La entidad `Employee` es autónoma. No hay grafos de objetos ni cascadas que gestionar.

Conclusión del análisis: El problema es **simple**, con bajo acoplamiento entre entidades y una lógica de negocio trivial. Por lo tanto, la solución debe priorizar la **simplicidad** y la **velocidad de implementación** sobre la flexibilidad extrema o el desacoplamiento estricto.

3. Fase 2: Selección del Patrón (¿Cuál es la mejor opción?)

Paso 2: Evaluar las alternativas

Para un CRUD simple, los patrones más comunes son **Active Record** y **Data Mapper**. Debes evaluarlos según los criterios del enunciado:

1. **Acoplamiento:** ¿Qué tan acoplado queda el dominio a la base de datos?
2. **Simplicidad de implementación:** ¿Cuál requiere menos clases y configuración?
3. **Adherencia a "Convención sobre Configuración":** ¿Cuál sigue las convenciones del framework sin necesidad de mapeos explícitos?

Comparación:

- **Active Record:** La entidad `Employee` contiene tanto los datos como los métodos de persistencia (`Save`, `Delete`, `Find`). Es directo, requiere una sola clase y se alinea con la convención de que la tabla `Employees` se mapea a la clase `Employee`. **Ventaja:** Simplicidad y rapidez. **Desventaja:** Acoplamiento alto (la entidad conoce la infraestructura).
- **Data Mapper:** Requiere separar la entidad pura (POCO) de una clase mapeadora (o usar el `DbContext` de EF Core como mapeador). **Ventaja:** Bajo acoplamiento. **Desventaja:** Más clases, más configuración, mayor complejidad inicial.

Decisión: Dado que el enunciado enfatiza la **simplicidad** y la **lógica elemental**, **Active Record** es la elección correcta. Data Mapper sería **sobreingeniería** para este contexto, ya que añadiría complejidad accidental sin beneficios claros en esta etapa temprana.

4. Fase 3: Implementación (¿Cómo lo construyo?)

Paso 3: Diseñar la entidad y la infraestructura

Ahora que has elegido Active Record, debes implementarlo en C# con Entity Framework Core. Sigue estos subpasos:

1. **Definir la entidad `Employee`:** Incluye las propiedades (`Id`, `Name`, `Department`) con sus anotaciones de datos (`[Key]`, `[Required]`).
2. **Agregar métodos de persistencia:** Implementa `Save`, `Delete` y un método estático `FindById`. Estos métodos recibirán el `DbContext` como parámetro (una forma de inyectar la dependencia sin acoplar completamente la entidad a una instancia global).
3. **Crear el `DbContext`:** Define la clase `AppDbContext` con el `DbSet<Employee>` y la configuración de conexión.
4. **Probar el flujo:** Escribe un pequeño programa principal que cree, lea, actualice y elimine un empleado, demostrando que el patrón funciona.

Código de referencia (coherente con la solución esperada):

```
using Microsoft.EntityFrameworkCore;
using System.ComponentModel.DataAnnotations;

// 1. Entidad de Dominio con comportamiento Active Record
public class Employee
{
  [Key]
  public int Id { get; set; }

  [Required]
  public string Name { get; set; }
  public string Department { get; set; }

  // 2. Metodo de persistencia (Active Record)
  public void Save(AppDbContext context)
  {
    if (this.Id == 0)
    context.Employees.Add(this);
    else
    context.Employees.Update(this);

    context.SaveChanges();
  }

  // 3. Metodo de eliminacion (Active Record)
  public void Delete(AppDbContext context)
  {
    context.Employees.Remove(this);
    context.SaveChanges();
  }

  // 4. Metodo de busqueda estatico (Active Record)
  public static Employee FindById(AppDbContext context, int id)
  {
    return context.Employees.Find(id);
  }
}

// 5. DbContext (Infraestructura)
public class AppDbContext : DbContext
{
  public DbSet<Employee> Employees { get; set; }

  protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
  {
    optionsBuilder.UseSqlServer("YourConnectionStringHere");
  }
}

// 6. Uso
class Program
{
  static void Main()
  {
    using var context = new AppDbContext();

    // Crear
    var emp = new Employee { Name = "John Doe", Department = "IT" };
    emp.Save(context);

    // Leer
    var found = Employee.FindById(context, emp.Id);
    Console.WriteLine($"Empleado: {found.Name}");

    // Actualizar
    found.Department = "HR";
    found.Save(context);

    // Eliminar
    found.Delete(context);
  }
}
```

5. Fase 4: Evaluación y Discusión (¿Qué pasa si el sistema evoluciona?)

Paso 4: Analizar las limitaciones y proponer una refactorización

El inciso (c) te pide pensar en el futuro. El sistema crece: `Employee` ahora tiene relaciones bidireccionales con `Manager` y `Project`, y requiere validaciones transaccionales multi-tabla. Debes argumentar por qué Active Record ya no es suficiente y proponer una estrategia de refactorización hacia Data Mapper.
Razonamiento esperado:

- **Limitación 1 (SRP):** Active Record viola el Principio de Responsabilidad Única porque la entidad `Employee` ahora tiene dos razones para cambiar: la lógica de negocio (relaciones, validaciones) y la lógica de persistencia (cómo guardar el grafo de objetos en múltiples tablas).
- **Limitación 2 (Ciclos de vida y cascadas):** Al tener relaciones bidireccionales, la gestión de qué se guarda primero, qué se elimina en cascada y cómo se mantiene la consistencia transaccional se vuelve caótica si está dentro de la entidad.
- **Limitación 3 (Cohesión):** La entidad se vuelve una clase "Dios" que mezcla responsabilidades, dificultando su testeo unitario (necesitas una base de datos para probar la lógica de negocio).

Estrategia de refactorización hacia Data Mapper:

1. **Extraer POCOs:** Crear clases de dominio puras (`Employee`, `Manager`, `Project`) sin métodos de persistencia (`Save`, `Delete`). Solo con propiedades y lógica de negocio.
2. **Crear mapeadores:** Implementar clases mapeadoras (o usar el `DbContext` de EF Core configurado con `EntityTypeConfiguration`) que se encarguen de traducir los objetos a tablas y viceversa.
3. **Inyectar dependencias:** Los mapeadores reciben la conexión a la base de datos por inyección, manteniendo el dominio ignorante de la infraestructura.
4. **Evaluar el impacto:** La separación de capas mejora (dominio testeable sin base de datos), pero el costo es un aumento significativo en el número de artefactos (interfaces, mapeadores, factories) y una curva de aprendizaje mayor para el equipo.

6. Resumen del Razonamiento

Fase	**Pregunta clave que debes responder**
Análisis	¿Cuál es la complejidad real del dominio? ¿Hay relaciones? ¿La lógica es trivial o rica?
Selección	¿Qué patrón minimiza el acoplamiento sin añadir complejidad innecesaria? ¿Se alinea con las convenciones del framework?
Implementación	¿Cómo traduzco el patrón a código limpio y funcional? ¿Qué responsabilidades tiene cada clase?
Evaluación	¿Qué limitaciones estructurales aparecen cuando el sistema evoluciona? ¿Cómo refactorizo sin romper la separación de capas?

7. Conclusión

Resolver un ejercicio de patrones de acceso a datos no es solo escribir código que funcione. Es un proceso de **toma de decisiones arquitectónicas**. Esta guía te ha mostrado cómo analizar el contexto, comparar alternativas, implementar la solución y anticipar sus limitaciones. Aplicar este mismo razonamiento a los ejercicios restantes te permitirá desarrollar un criterio profesional para seleccionar el patrón adecuado en cada situación.